

MAKERERE



UNIVERSITY

COLLEGE OF COMPUTING AND INFORMATION SCIENCES

DEPARTMENT OF NETWORKS

COURSE: BACHELOR OF SCIENCE IN SOFTWARE ENGINEERING

COURSE UNIT: DATA STRUCTURES AND ALGORITHMS

COURSE CODE: CSC 2100

COURSEWORK TITLE: REAL LIFE APPLICATION OF STACKS AND QUEUES

GROUP: B2

GROUP MEMBERS

NAME	REGISTRATION NUMBER
SSEKIJOBA FRANCIS	25/U/04324/PS
NASSUUNA PATIENCE R	25/U/04295/PS
NUWAGABA DUNCAN	25/U/0392
OYET ANTHONY ELIZEOUS	25/E/04316/PS
BIDAL DANIEL	25/E/04188/PS

Implementation Report: LIFO & FIFO Data Structures

1. Our Two Applications

Following the brief's implementation matrix, we chose the pairing where the **Stack is built in C using an array** and the **Queue is built in Java using a linked list**. Both programs are menu-driven: they keep showing a menu and running the user's choice until the user exits. In both, the data structure code is kept separate from the main program, and no built-in stack or queue library is used.

	Project A: Stack (LIFO)	Project B: Queue (FIFO)
Application	Popup Manager	Call Centre Support Queue
Language	C	Java
Underlying structure	Array	Linked list
Folder	Project_A_Stack	Project_B_Queue

Project A: Popup Manager. This models popup windows stacking on top of each other on a PC screen. Each new popup opens in front and covers the ones behind it, and the only way to see the previous popup is to close the front one with its [x] or the [-] button. The most recently opened popup is therefore always the first one closed, which is Last In, First Out.

Project B: Call Centre Support Queue. This models a helpdesk call centre. Callers join the back of a waiting line, and each time an agent is free they answer the caller who has waited longest. The first caller in is the first caller answered, which is First In, First Out. Each call stores the caller's name, the issue type and a timestamp taken when the call is created.

2. Project A: How push and pop Work on the Array (C)

The stack is a `PopupStack` structure with two parts: an array `popups` that holds up to 5 popups (`MAX_POPUPS`), and an integer `top` that is the index of the front popup. `top` is -1 when no popups are open.

- **Push (open a popup):** we first check `isFull` (`top == MAX_POPUPS - 1`) and refuse with a warning if so. Otherwise we increase `top` by 1 first, and then store the new popup at `popups[top]`.
- **Pop (click [x]):** we first check `isEmpty` (`top == -1`) and warn if so. Otherwise we **copy out the popup at `popups[top]` first**, and then decrease `top` by 1. The old slot is not erased: it is now above `top`, so it counts as free and the next push overwrites it. The program then uses `peek` to report which popup has been revealed behind it.
- **Peek (view front):** reads `popups[top]` without changing `top`.
- **isEmpty, isFull and size:** size is `top + 1`, because array indexes start at 0.
- **Show screen:** loops from `top` down to index 0, so the front popup prints first. Only the front popup shows the [x] button, because only it can be closed.

Edge cases: closing or viewing with no popups open prints a warning; opening a 6th popup is refused with a "screen is full" warning before the user types anything; non-numbers and unknown menu choices are rejected; text longer than the title or message limit is shortened without spilling into the next prompt; blank text gets a default; and end of input exits cleanly.

Code organisation: Ui.h and Ui.c hold the structure and every stack operation (initStack, push, pop, peek, isEmpty, isFull, size, display). Main.c only handles the menu and user input.

3. Project B: How enqueue and dequeue Work on the Linked List (Java)

The queue is a chain of CallNode objects. Each node holds one Call and a next reference to the node behind it. CallCenterQueue keeps three fields: front (the head, the next caller to answer), rear (the tail, the newest caller) and size. When the queue is empty, front and rear are both null.

- **Enqueue (receiveCall):** we create a new CallNode for the call. If the queue is empty, both front and rear point to it. Otherwise we **attach it to the current rear.next and then move rear forward** to the new node. size goes up by 1. Nodes only ever join at the tail.
- **Dequeue (answerCall):** if the queue is empty we print a warning and stop. Otherwise we save the call held by the front node and **reassign front to front.next**, so the old head drops off the list. If front is now null, the queue has just become empty, so rear is also set to null; without this, rear would keep pointing at a removed node. size goes down by 1.
- **Front (viewFront):** returns the call at front without removing it. **isEmpty** is true when front == null, and every other operation uses it as its empty check.
- **Show waiting calls (printFullQueue):** walks the list from the front using a temporary pointer, so the queue's own front is never moved or lost.

Edge cases: answering or viewing with no calls waiting prints "No calls in the queue." instead of causing a null pointer exception; showing an empty queue prints a message; a blank caller name or issue is rejected; and invalid menu input is rejected. No isFull check is needed, because a linked list grows as needed.

Code organisation: Call, CallNode and CallCenterQueue hold the data and all queue logic. Main only handles the menu and user input.

4. Running the Applications

For the queue implementation i.e. The call center, when the application loads the agent receives the options as shown in the figure below and when it's empty the application gives a friendly message indicating an empty queue rather than breaking down as shown in Figure 1.

The queue displays the current waiting calls and the agent always chooses first the caller who made a call first following that order, consequently, the last caller is served last.

For the stack, as shown in Figure 4, the user is always showed when the stack is full and cannot add any other items thereafter unless they pop out some.

```
Project_B_queue 55 if (!input.hasNextLine()) return;
                   stdin.readln();
Run Main x
C:\Users\user\.jdk\openjdk-26\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.4\lib\idea_rt.jar=62567" -Dfile.encoding=UTF-8

=== Call Centre Support Queue ===
1. Receive a call
2. Answer next call
3. View next caller
4. Show waiting calls
0. Exit
Choice: 4
The queue is currently empty.

=== Call Centre Support Queue ===
1. Receive a call
2. Answer next call
3. View next caller
4. Show waiting calls
0. Exit
Choice: |
```

Figure 1 : Initial display when the user first runs the program and no queued calls available

```
Project_B_queue 55 if (!input.hasNextLine()) return;
                   stdin.readln();
Run Main x
Choice: 1
Caller name: Oyet
Issue type: Complaint on charges
Received: Oyet | Queue size: 2

=== Call Centre Support Queue ===
1. Receive a call
2. Answer next call
3. View next caller
4. Show waiting calls
0. Exit
Choice: 4

--- Current Call Queue (2 waiting) ---
1. Duncan - Inquiry on balance
2. Oyet - Complaint on charges
-----
```

Figure 2 : Demo of Oyet being added last in queue and has to wait for Duncan to be worked on first

```
PS C:\Users\user\Desktop\test\Project_A_Stack> .\main.exe

=== Popup Manager ===
1. Open a popup      (push)
2. Close front popup [x] (pop)
3. View front popup  (peek)
4. Show screen
0. Exit
Choice: |
```

Figure 3: For the stack, demo of first view when loaded

```
PS C:\Users\user\Desktop\test> .\main.exe

=== Popup Manager ===
1. Open a popup      (push)
2. Close front popup [x] (pop)
3. View front popup  (peek)
4. Show screen
0. Exit
Choice: 1
WARNING: Screen is full (5 popups). Close one first.

=== Popup Manager ===
```

Figure 4: View when the stack is full, and the user is asked to remove items to proceed